Lesson notes · for you

Recursion

Programming · Level 4 · Lesson 2

Time	Materials
20 min	A phone with this lesson downloaded; Three coins of different sizes for Hanoi (optional)

Learning goals

- Write recursive functions with a base case and a recursive case.
- Follow the call stack and know its limits.
- Speed up repeated work with memoisation.

Activity: Activity: The human call stack

1. Warm-up (3 min): Five people stand in a line. Each asks the person behind, “How many are behind you?”, until the last says “none”. Answers pass back up the line.
2. Act (7 min): Act out fact(4): each person is one call, writes their n, and waits until the next person hands back an answer. The last person (n = 1) is the base case.
3. Count (7 min): Draw fib(5)’s call tree on paper and circle every call that repeats work done elsewhere. How many times does fib(2) appear?
4. Fix (3 min): Add a shared sheet of answers already worked out. Redraw the tree, crossing out calls you can look up. How many are left?

A paper call tree makes repeated work visible before any code runs.

Discussion

- When is a loop better than recursion?
- Why does fib repeat so much work?
- What problems around you are made of smaller copies of themselves?

Check yourself

- Write the base case and the recursive case of a function that adds up a list.

Try it this week

Stand five friends in a line. Each asks the next, “How many people are behind you?”, until the last says “none”. The answers come back up the line: that is recursion, base case included.

Worksheet

Recursion

Programming · Level 4 · Lesson 2

Name _____ Class _____ Date _____

1 **fib(30) is painfully slow, because it works out the same values again and again. What fixes it best?**

◯ Run it on a faster phone ◯ Remember the answers in a dictionary ◯ Remove the base case

2 **Write digit_sum(n) recursively: it returns the sum of the digits of a whole number n ≥ 0. Base case: a single digit is its own sum. Recursive case: the last digit plus digit_sum of the rest.**

```
def digit_sum(n):
    # base case: one digit (n < 10)
    # recursive case: n % 10 + digit_sum(n // 10)
    return 0
```

Write your program:

3 **Put the recursive factorial in order, then print fact(5). One line doesn't belong.**

Number the lines in order (1, 2, 3...). Cross out any line that doesn't belong.

☐ `def fact(n):`

☐ `return n * fact(n)`

☐ `if n == 1:`

☐ `print(fact(5))`

☐ `return 1`

☐ `return n * fact(n - 1)`

4 **countdown(3) should print 3, 2, 1 and stop, but it never reaches its base case. Which line is wrong?**

Circle the line with the mistake. Write it correctly:

```
1  def countdown(n):
2      if n == 10:
3          return
4      print(n)
5      countdown(n - 1)
6
7  countdown(3)
```

5 **The Towers of Hanoi take $2^n - 1$ moves for n discs. How many moves for 5 discs?**

◯ 25 ◯ 31 ◯ 32

✎ **Where do you see a problem made of smaller copies of itself (folders inside folders, a family tree, a tournament)? Describe its base case and its recursive case.**